



Network Programming - 19

Standalone asio 채팅 서버

afewhee@gmail.com





- asio 개요
- 개발 환경 설정
- 채팅 서버 구조
- 채팅 서버 코드
- 채팅 클라이언트 코드
- 메시지 처리와 정리





- Standalone asio 기본 구조 이해
- TCP 채팅 서버 / 클라이언트 실행
- Winsock 코드와 asio 코드 비교
- C++ 객체 기반 소켓 처리 이해





- Winsock → Windows 소켓 API 직접 사용
- IOCP → Windows 고성능 비동기 I/O 모델
- asio → C++ 객체 기반 네트워크 라이브러리
- 동기식 asio 기반 TCP 통신





- asio
 - ◆ Asynchronous Stream I/O
 - ◆ C++ 네트워크 / 비동기 I/O 라이브러리
 - ◆ 운영체제(OS)의 비동기 I/O 메커니즘을 추상화
- 지원 기능
 - ◆ TCP / UDP
 - ◆ Timer
 - ◆ Serial Port
 - ◆ 동기, 비동기 I/O
- 특징
 - ◆ C++ 객체 기반
 - ◆ 크로스 플랫폼 구조
 - ◆ boost.asio / Standalone asio 제공





- boost.asio
 - ◆ boost 라이브러리 포함 버전
 - ◆ namespace: boost::asio
 - ◆ include: `<boost/asio.hpp>`
- Standalone asio
 - ◆ boost 없이 사용 가능한 독립 버전
 - ◆ namespace: asio
 - ◆ include: `<asio.hpp>`
 - ◆ 전처리기: ASIO_STANDALONE





Winsock

WSAStartup

socket

bind

listen

accept

recv

send

closesocket

WSACleanup

asio

io_context

tcp::acceptor

tcp::socket

acceptor.accept()

read_until()

write()

socket.close()





- Standalone asio 다운로드
 - ◆ 공식 사이트: <https://think-async.com/asio/>
- 다운로드 위치
 - ◆ SourceForge
- 사용 파일
 - ◆ asio-1.36.0.zip
- 주의
 - ◆ boost_asio_1_36_0.zip 아님
 - ◆ asio-1.36.0.zip 사용





- C++ 언어 표준
 - ◆ ISO C++17 표준 (/std:c++17)
- 추가 포함 디렉터리
 - ◆ asio-1.36.0\include
- 전처리기 정의
 - ◆ asio_STANDALONE
 - ◆ _WIN32_WINNT=0x0A00
 - ◆ _CRT_SECURE_NO_WARNINGS
- 추가 라이브러리
 - ◆ ws2_32.lib





- Standalone asio 사용 지정 → ASIO_STANDALONE
- Windows API 대상 버전 지정 → _WIN32_WINNT
- Windows 소켓 라이브러리 → ws2_32.lib

```
#define ASIO_STANDALONE
#define _WIN32_WINNT 0x0A00

#include <asio.hpp>
#include <iostream>

#pragma comment(lib, "ws2_32.lib")
```





Server

클라이언트 접속 대기
클라이언트 목록 관리
클라이언트별 수신 thread 생성
수신 메시지 전체 클라이언트 전송

Client

서버 접속
사용자 입력 송신
서버 메시지 수신





- `asio::io_context` ← asio I/O 작업 관리 객체
- `tcp::acceptor` ← 클라이언트 접속 대기 객체
- `tcp::socket` ← 클라이언트와 통신하는 TCP 소켓
- `std::vector` ← 접속 클라이언트 목록
- `std::mutex` ← 클라이언트 목록 동시 접근 보호





- `asio::io_context` ← asio I/O 작업 관리 객체
- `tcp::endpoint` ← 서버 IP 주소와 포트 정보
- `tcp::socket` ← 서버와 통신하는 TCP 소켓
- `std::thread` ← 서버 메시지 수신 전용 thread





- io_context 생성
- acceptor 생성
- socket 생성

- accept로 접속 대기
- 클라이언트 목록 추가
- 클라이언트 thread 생성
- read_until로 메시지 수신
- Broadcast로 전체 전송

- 종료 시 목록 제거





- `io_context` 생성
- `socket, endpoint` 생성
- `connect`로 서버 접속
- 수신 `thread` 생성
- 사용자 입력
- `write`로 서버 전송
- `read_until`로 서버 메시지 수신





4.1 서버 include

```
#define asio_STANDALONE
#define _WIN32_WINNT 0x0A00

#include <asio.hpp>
#include <iostream>
#include <vector>
#include <string>
#include <thread>
#include <mutex>
#include <memory>
#include <algorithm>

#pragma comment(lib, "ws2_32.lib")

using asio::ip::tcp;
```





4.2 서버 전역 데이터

```
// 접속 중인 클라이언트 socket 목록  
std::vector<std::shared_ptr<tcp::socket>> g_clients;  
  
// 여러 thread에서 g_clients 접근 보호  
std::mutex g_clientsMutex;  
  
// thread 함수에서 socket 생명 주기 유지  
shared_ptr<>
```





4.3 서버 main 구조

```
int main() {
    try {
        asio::io_context ioContext;
        tcp::acceptor acceptor(ioContext
                                , tcp::endpoint(tcp::v4(), 60000) );

        std::cout << "Chat Server Start\n";
        std::cout << "Port: 9000\n";

        while (true) {
            auto socket = std::make_shared<tcp::socket>(ioContext);
            acceptor.accept(*socket);
            {
                std::lock_guard<std::mutex> lock(g_clientsMutex);
                g_clients.push_back(socket);
            }
            std::thread(WorkClient, socket).detach();
        }
    }
    catch (const std::exception& e) {
        std::cout << "Server Error: " << e.what() << "\n";
    }
    return 0;
}
```





```
asio::io_context ioContext;  
io_context
```

- asio::io_context → I/O 작업 관리 객체
- 역할
 - ◆ socket 생성 기준
 - ◆ acceptor 생성 기준
 - ◆ I/O 작업 관리





```
tcp::acceptor acceptor(  
    ioContext,  
    tcp::endpoint(tcp::v4(), 60000)  
);
```

- tcp::acceptor
 - ◆ 서버 소켓 역할
 - ◆ tcp::v4() ← IPv4 사용
 - ◆ 9000 ← 서버 포트 번호
- Winsock 대응
 - ◆ socket → bind → listen





```
auto socket =  
std::make_shared<tcp::socket>(ioContext);  
  
acceptor.accept(*socket);  
acceptor.accept()
```

- 클라이언트 접속까지 대기
- 접속 성공
 - ◆ socket 객체에 연결 정보 저장
- 동기식 함수
 - ◆ 접속 전까지 다음 코드 진행 없음





```
{  
    std::lock_guard<std::mutex> lock(g_clientsMutex);  
    g_clients.push_back(socket);  
}
```

- 접속 클라이언트 저장 → g_clients
- lock_guard ← 동기화. 함수 블록 동안 mutex 잠금





```
std::thread(WorkClient, socket).detach();
```

- 클라이언트 thread ← 클라이언트 1명당 thread 1개
- main thread ← accept 반복
- client thread ← 해당 클라이언트 메시지 수신 처리
- Detach ← thread 독립 실행





4.9 WorkClient

```
void WorkClient(std::shared_ptr<tcp::socket> socket) {
    try {
        Broadcast("[System] Client joined.\n");
        asio::streambuf buffer;
        while (true) {
            asio::read_until(*socket, buffer, '\n');

            std::istream is(&buffer);
            std::string message;
            std::getline(is, message);

            if (message == "/quit") {
                break;
            }
            std::string sendMessage = "[Client] " + message + "\n";
            std::cout << sendMessage;
            Broadcast(sendMessage);
        }
    }
    catch (const std::exception& e) {
        std::cout << "Client disconnected: " << e.what() << "\n";
    }
    RemoveClient(socket);
    Broadcast("[System] Client left.\n");
}
```





```
asio::streambuf buffer;
```

- asio::streambuf
 - ◆ asio 수신 버퍼
 - ◆ read_until 사용 시
 - ◆ 구분자까지 받은 데이터 저장
 - ◆ istream 연결 가능
 - ◆ std::getline으로 문자열 추출 가능





```
asio::read_until(*socket, buffer, '\n');
```

- read_until
 - ◆ 지정 문자까지 데이터 수신
- 예제 구분자
 - ◆ '\n'
- 주의
 - ◆ TCP는 메시지 단위 없음
 - ◆ 문자열 끝 구분자 필요





```
std::istream is(&buffer);  
  
std::string message;  
std::getline(is, message);
```

- streambuf ← 수신 데이터 저장
- istream ← streambuf에서 문자열 읽기
- getline ← '\n' 전까지 한 줄 추출





- 클라이언트 socket 검색
- 열려 있으면 close
- g_clients에서 제거

```
void RemoveClient(std::shared_ptr<tcp::socket> socket)
{
    std::lock_guard<std::mutex> lock(g_clientsMutex);

    auto it = std::find(g_clients.begin(), g_clients.end(), socket);

    if (it != g_clients.end())
    {
        if ((*it)->is_open())
        {
            (*it)->close();
        }

        g_clients.erase(it);
    }
}
```





5.1 클라이언트 include

```
#define asio_STANDALONE
#define _WIN32_WINNT 0x0A00

#include <asio.hpp>
#include <iostream>
#include <string>
#include <thread>

#pragma comment(lib, "ws2_32.lib")

using asio::ip::tcp;
```





5.2 클라이언트 main 구조

```
int main() {
    try {
        asio::io_context ioContext;
        tcp::socket socket(ioContext);
        tcp::endpoint endpoint(asio::ip::make_address("127.0.0.1"), 60000);
        socket.connect(endpoint);

        std::cout << "Connected to server.\n";
        std::cout << "Input message. /quit to exit.\n";

        std::thread recvThread(ReceiveThread, &socket);
        while (true) {

            std::string message;
            std::getline(std::cin, message);
            message += "\n";
            asio::write(socket, asio::buffer(message));
            if (message == "/quit\n")
            {
                break;
            }
        }
        socket.close();
        if (recvThread.joinable()) {
            recvThread.join();
        }
    }
    catch (const std::exception& e) {
        std::cout << "Client Error: " << e.what() << "\n";
    }

    return 0;
}
```





```
asio::io_context ioContext;  
tcp::socket socket(ioContext);
```

- tcp::socket
 - ◆ 서버와 통신하는 객체
- 클라이언트 역할
 - ◆ 서버 접속
 - ◆ 메시지 송신
 - ◆ 메시지 수신
 - ◆ 연결 종료





```
tcp::endpoint endpoint(  
    asio::ip::make_address("127.0.0.1"),  
    60000  
);
```

- endpoint
 - ◆ 서버 주소 정보 설정
 - ◆ IP 주소 + Port 번호





```
socket.connect(endpoint);
```

- connect
 - ◆ 서버 접속 요청
 - ◆ 동기식 함수
 - ◆ 접속 성공 또는 실패까지 대기
 - ◆ Winsock 대응 connect 함수





```
std::thread recvThread(ReceiveThread, &socket);
```

- 수신 thread
 - ◆ 서버 메시지 수신 전용
 - ◆ main thread: 송신
 - ◆ receive thread: 수신





5.7 ReceiveThread

```
void ReceiveThread(tcp::socket* socket)
{
    try
    {
        asio::streambuf buffer;

        while (true)
        {
            asio::read_until(*socket, buffer, '\n');

            std::istream is(&buffer);
            std::string message;
            std::getline(is, message);

            std::cout << message << "\n";
        }
    }
    catch (const std::exception& e)
    {
        std::cout << "Disconnected from server: " << e.what() << "\n";
    }
}
```





- read_until
 - ◆ 서버 메시지 수신
- getline
 - ◆ 한 줄 메시지 추출
- 화면 출력
- 예외 발생
- 서버 종료
- 연결 끊김
- socket close





```
while (true)
{
    std::string message;
    std::getline(std::cin, message);

    message += "\n";

    asio::write(socket, asio::buffer(message));

    if (message == "/quit\n")
    {
        break;
    }
}
```





- 사용자 입력
 - ◆ `std::getline`
- 메시지 끝 '\n' 추가
- 서버 송신
 - ◆ `asio::write`
- 종료 확인





```
socket.close();  
  
if (recvThread.joinable())  
{  
    recvThread.join();  
}
```

- **socket.close**
 - ◆ 서버와 연결 종료
- **join**
 - ◆ 수신 thread 종료 대기
- **주의**
 - ◆ 수신 thread가 read_until 대기 중이면
 - ◆ socket close 후 예외 발생 가능





- TCP 메시지 경계
- TCP
 - ◆ 바이트 스트림
- 문제
 - ◆ send 1번 = recv 1번 아님
 - ◆ 메시지 단위 보장 없음
- 해결 방식
 - ◆ 구분자 방식
 - ◆ 길이 헤더 방식





- 구분자 방식
 - ◆ 메시지 끝에 특정 문자 추가
 - ◆ 예
 - HelloWn
 - HiWn
 - /quitWn
- 수신
 - ◆ `read_until(socket, buffer, 'Wn')`
- 장점
 - ◆ 문자열 채팅 구현 단순
- 단점
 - ◆ 바이너리 패킷 처리 부적합





- 길이 헤더 방식
 - ◆ 패킷 앞에 데이터 크기 추가
- 구조
 - ◆ [4바이트 길이] [데이터]
- 장점
 - ◆ 바이너리 데이터 처리 가능
 - ◆ 구조화된 패킷 처리 가능
- 단점
 - ◆ 구현 복잡
 - ◆ partial receive 처리 필요





- 동기식 asio
 - ◆ 함수 호출 위치에서 대기
- accept
 - ◆ 접속까지 대기
- read_until
 - ◆ 수신까지 대기
- write
 - ◆ 송신 완료까지 대기
- 장점
 - ◆ 코드 흐름 단순
- 단점
 - ◆ 다중 접속 처리 시 thread 필요





- 비동기 asio
 - ◆ I/O 작업 요청 후 즉시 반환
- 주요 함수
 - ◆ Async_accept
 - ◆ Async_read
 - ◆ Async_write
- 완료 처리
 - ◆ handler 호출
- 실행 루프
 - ◆ `io_context.run()`
- Windows 내부 관점
 - ◆ Overlapped I/O / IOCP 계열 추상화





- 동기식 asio
 - ◆ thread 기반 다중 클라이언트 채팅 서버
- 목적
 - ◆ asio 객체 구조 이해
 - ◆ Winsock 코드와 비교
 - ◆ C++ 네트워크 라이브러리 경험





- asio 함수
 - ◆ 실패 시 `std::exception` 발생 가능
- 주요 상황
 - ◆ 서버 미실행
 - ◆ 접속 실패
 - ◆ 클라이언트 강제 종료
 - ◆ 서버 종료
 - ◆ read 실패
 - ◆ write 실패
- 처리 방식
 - ◆ try-catch
 - ◆ `socket.close`
 - ◆ 클라이언트 목록 제거





- asio
 - ◆ C++ 기반 네트워크 I/O 라이브러리
 - ◆ Standalone asio boost 없이 사용 가능
- TCP 서버
 - ◆ io_context
 - ◆ tcp::acceptor
 - ◆ tcp::socket
 - ◆ accept
- TCP 클라이언트
 - ◆ endpoint
 - ◆ connect
 - ◆ read_until
 - ◆ write
- 채팅 서버
 - ◆ thread / mutex / client list

